pfi

Python for Fixed Income

COURSE 5 · ADVANCED MACHINE LEARNING

AdaBoost

WEEK 2 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

How AdaBoost works

1. Fit a stump

one question, every row weighted equally

2. Give it a vote

bigger for a lower weighted error

3. Reweight

rows it missed count more in the next round

4. Repeat

the answer is the weighted vote of all the stumps

You control

n_estimators, learning_rate, and how deep each member may be.

What comes out

A fitted AdaBoostClassifier: predict_proba is the model's score, estimator_weights_ the votes.

Side by side, or one after another

	Bagging and forests (week 1)	Boosting (week 2)
Rows each tree sees	its own bootstrap sample	every row, with weights
Order	independent	one after another
The trees	deep and unstable	small and weak: stumps
Combined by	a plain average	a weighted vote
What it fixes	the noise of one deep tree	the bias of one small tree

Each new tree is aimed at the rows the trees before it got wrong.

How a stump works

1. Try every question

each column, each threshold

2. Keep the purest

the split with the lowest Gini impurity

3. Two groups

one question, two leaves

4. Answer with a rate

each leaf returns its default rate

You control

The depth (a stump is max_depth=1) and the seed that breaks ties.

What comes out

One question, two rates, and a score that takes two values.

A stump: a tree with one question

The stump asks

```
PAY_0 <= 1.5
```

16.51%

default rate, yes side (16,115 rows)

70.13%

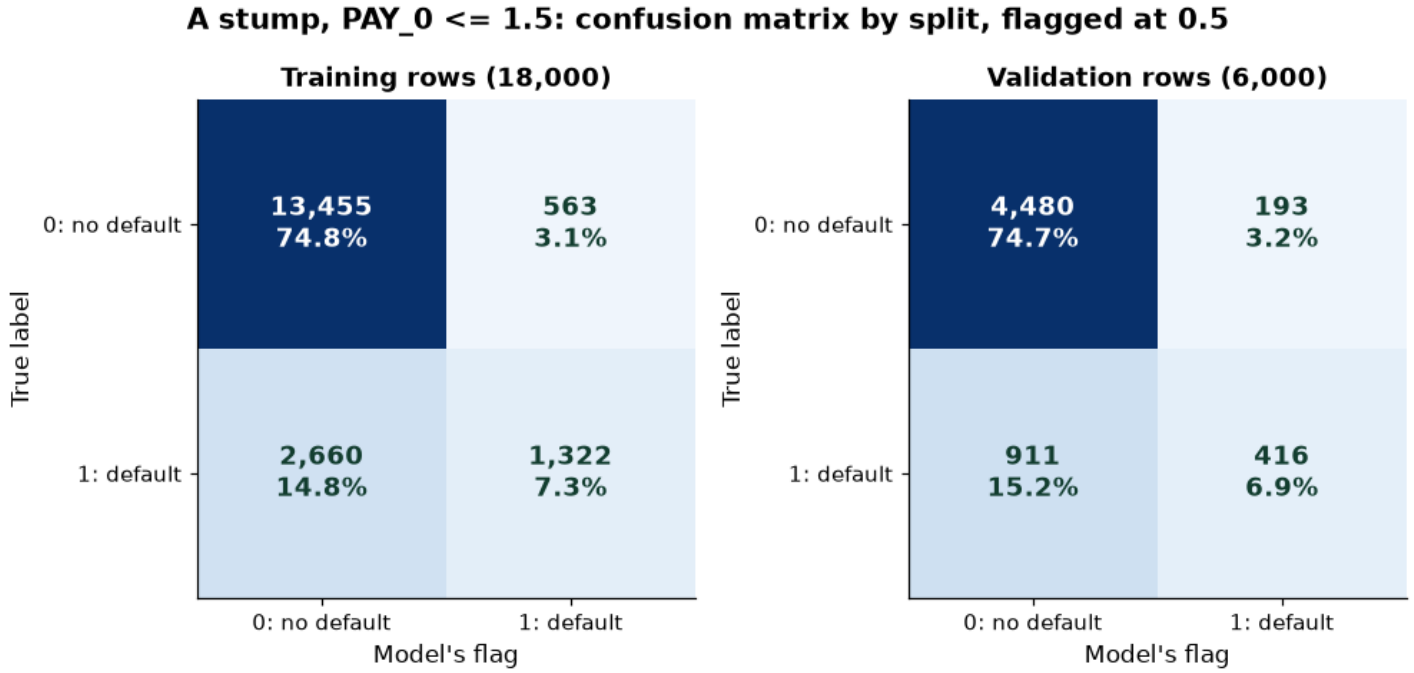
default rate, no side (1,885 rows)

0.6361

validation ROC AUC

Weak on purpose. AdaBoost uses two hundred of them.

The standard view: one heatmap per split



One question cannot memorise rows: the two panels look alike.

How to read it

True label down the side, the model's flag at 0.5 across the top. Each cell: a count and its share of that split's rows.

3,223 off the diagonal

The training panel's two wrong cells are the rows round one will reweight.

Round one by hand

1. Fit

a stump on the weighted rows

2. Error

share of the weight on missed rows

3. Weight

ln((1 - e) / e), samme_alpha

4. Reweight

missed rows times e^alpha, then / sum

3,223

rows missed of 18,000

0.1791

weighted error e

1.5228

stump weight

Every row starts at 1/n, so round one's weighted error is the share of rows missed.

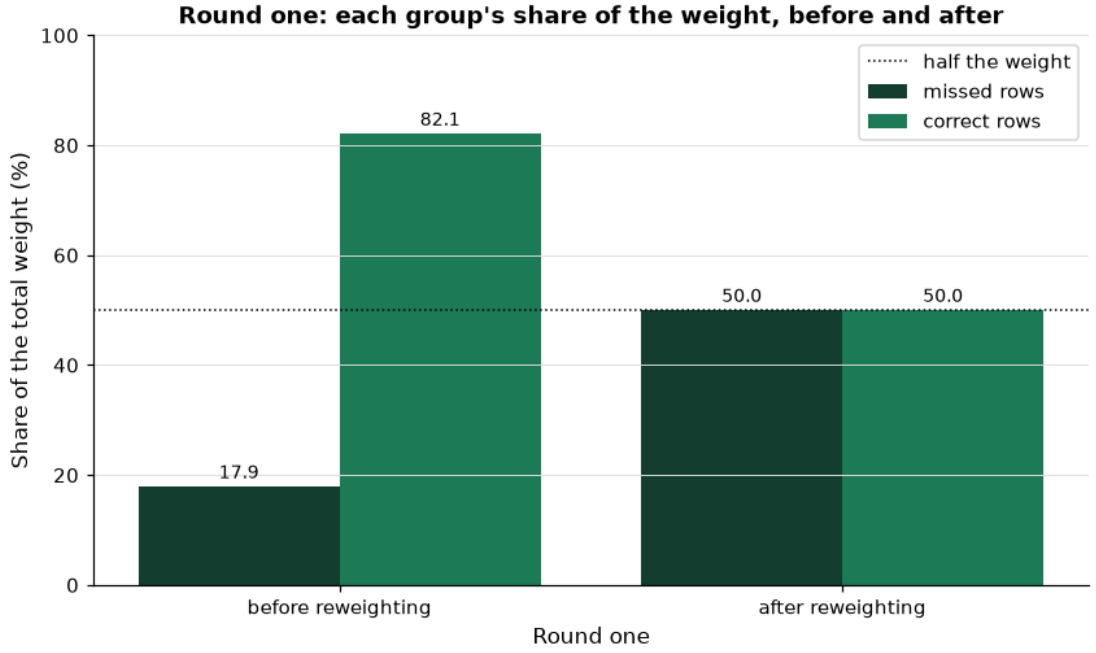
After reweighting, the missed rows carry half the weight

0.1791
missed rows' weight, before

0.5000
after

4.5849
missed over correct

```
e  x  (1 - e) / e  =  1 - e
the missed rows now weigh what the correct rows weigh
```



On the new weights the last stump is a coin toss. The next stump has to find something else.

Three rounds by hand, matched to scikit-learn

Round	Question	Weighted error	Hand weight	scikit-learn
1	PAY_0 <= 1.5	0.1791	1.5228	1.5228
2	PAY_AMT1 <= 1223.5	0.4128	0.3526	0.3526
3	PAY_3 <= 1.5	0.4356	0.2592	0.2592

Equal within 1e-12. AdaBoostClassifier is these four steps, repeated.

```
ada_3.estimator_weights_     # each stump's weight in the vote
```

Two hundred stumps

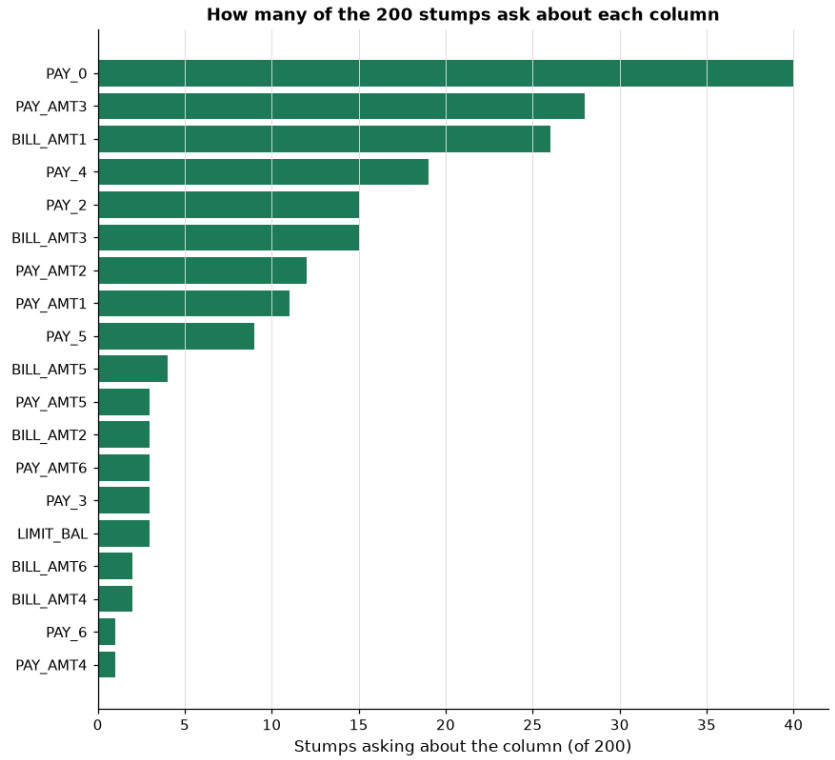
0.7708
validation ROC AUC

0.5321
average precision

0.6361
one stump

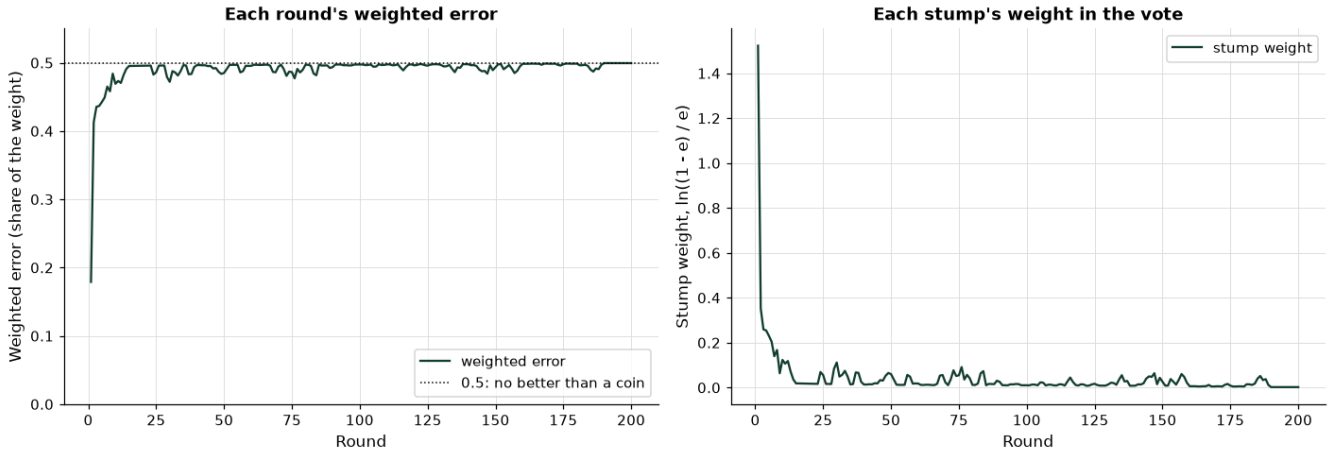
The stumps ask about 19 of the 19 columns

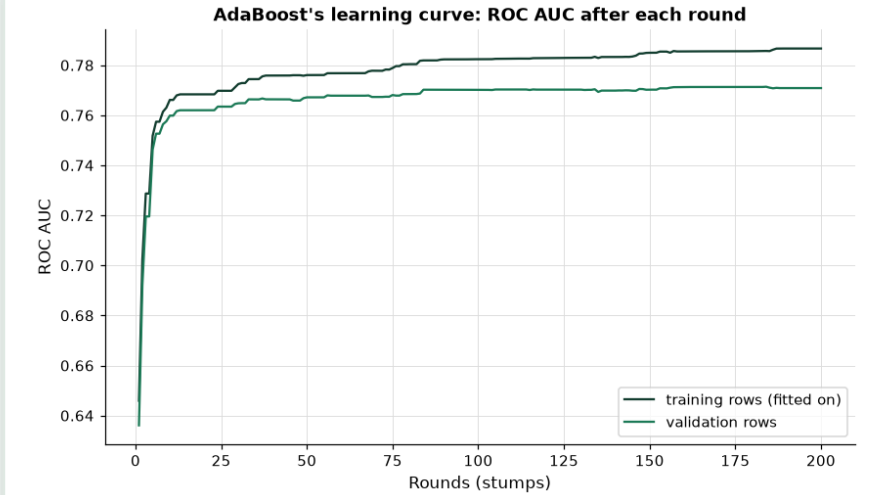
```
PAY_0 40 times
PAY_AMT3 28 times
BILL_AMT1 26 times
```



A sum of one-column steps: it bends along each column, never two at once.

Round by round: the error, the weight, and the learning curve





0.7867

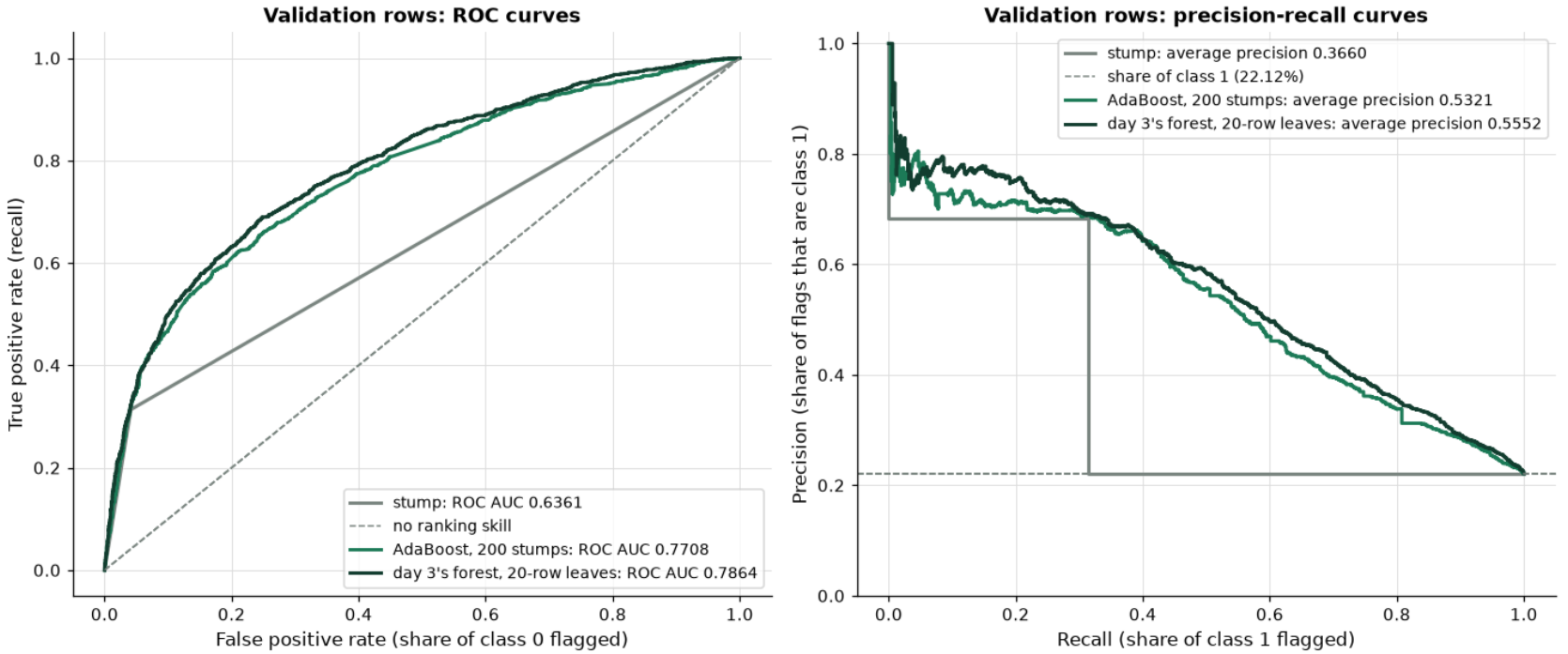
training rows, after 200 rounds

0.7708

validation rows

Later stumps are barely better than a coin and get a small say. The two curves stay close: little is memorised.

AdaBoost beside day 3's forest: validation rows



Forest 0.7864 ROC AUC, 0.5552 average precision; AdaBoost 0.7708 and 0.5321. The stump's curve has one corner.

How the comparison works

1. Cut five folds

of the training rows only

2. Fit on four

score on the fifth

3. Score every round

staged_decision_function: one fit per fold

4. Average

the highest point of a curve is the choice

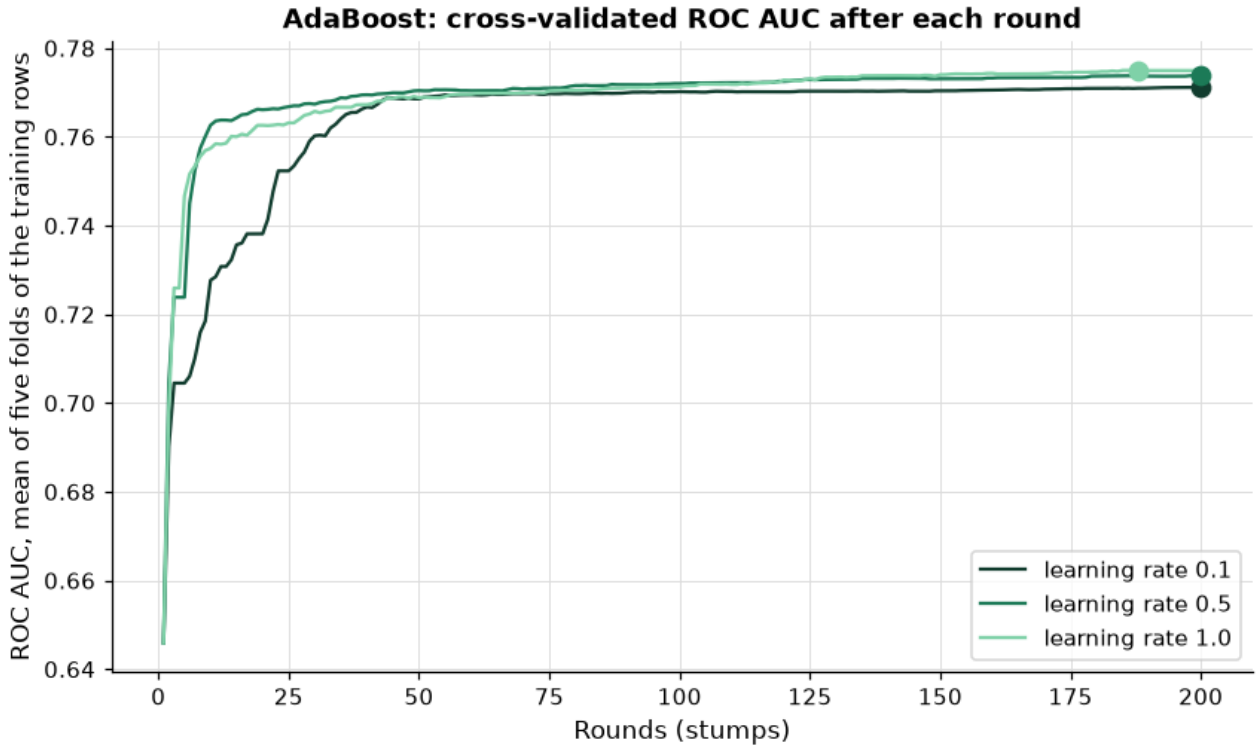
You control

The folds, the settings compared, and the score.

What comes out

A held-out ROC AUC curve per round for each setting.

The learning rate and the number of rounds



Rounds	0.1	0.5	1.0
10	0.7277	0.7626	0.7574
50	0.7686	0.7704	0.7688
200	0.7712	0.7738	0.7749

Rate 1.0 peaks at round 188, 0.7750. Five folds of the training rows; the validation rows play no part.

The learning rate mostly decides how many rounds it takes, not where the curves end.

How the vote check works

1. Collect the votes

each stump says +1 or -1, with its weight

2. Weighted average

between -1 and +1

3. Logistic curve

1 / (1 + e^(-2 x vote)) is predict_proba

4. Ten score groups

mean score against the actual default rate

You control

The number of score groups.

What comes out

A table and a chart of default rate against mean score, read against the diagonal.

A weighted vote, not a frequency

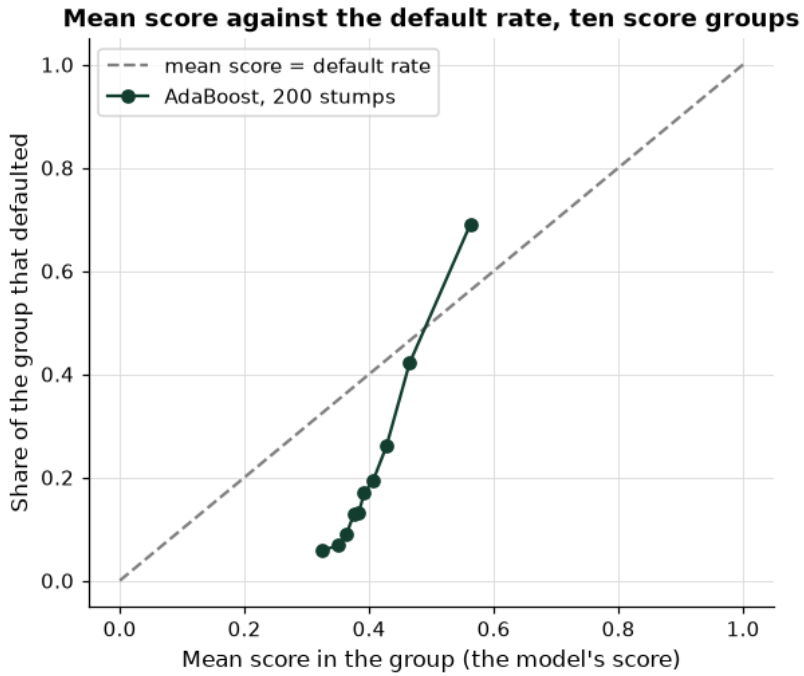
0.1192
lowest possible

0.8808
highest possible

0.4051
mean output

0.2212
default rate

Score group	Mean score	Default rate
1	0.3244	0.0583
5	0.3832	0.1308
10	0.5623	0.6900



It ranks well; its level is not a share of defaults. The model's score, never a probability. Vote -0.4803 to 0.3243.

How the weights reading works

1. Run 50 rounds

by hand, keeping the row weights

2. Divide by the start

1/n: a multiple of where each row began

3. Group the rows

1, 2, 3, and 4 times heavier

4. Describe them

default rate, delays, credit limit

You control

The number of rounds and the cut points of the groups.

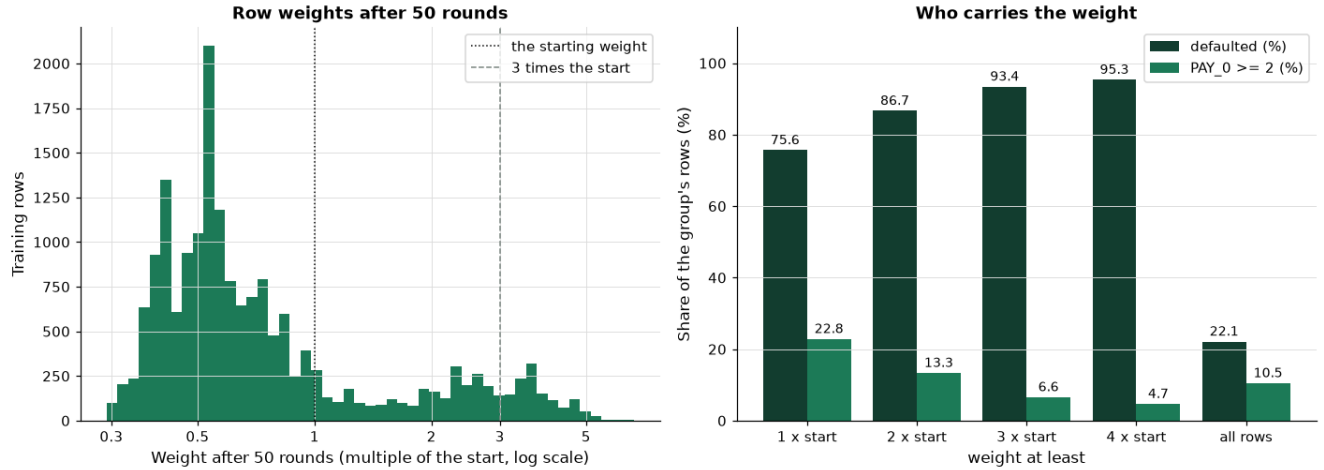
What comes out

A weight for every training row and a description of the heavy ones.

The rows that carry the weight after 50 rounds

Weights run from 0.29 to 6.63 times where they started. The rows at three times or more:

Rows	Count	Share of weight	Default rate	PAY_0 >= 2
Weight 3 x start or more	1,309	27.8%	93.4%	6.6%
All training rows	18,000	100.0%	22.1%	10.5%



Mostly accounts that defaulted while their latest status looked current: where the model struggles in this file, not a judgment of the accounts.

Every setting of AdaBoostClassifier

Setting	Default	Tune first?
estimator	None	tune first
n_estimators	50	tune first
learning_rate	1.0	tune first
random_state	None	

Member setting	Default	Tune?
max_depth	1 (set by AdaBoost)	tune first
min_samples_leaf	1	rarely
max_leaf_nodes	None	rarely
class_weight	None	a choice
ccp_alpha	0.0	rarely

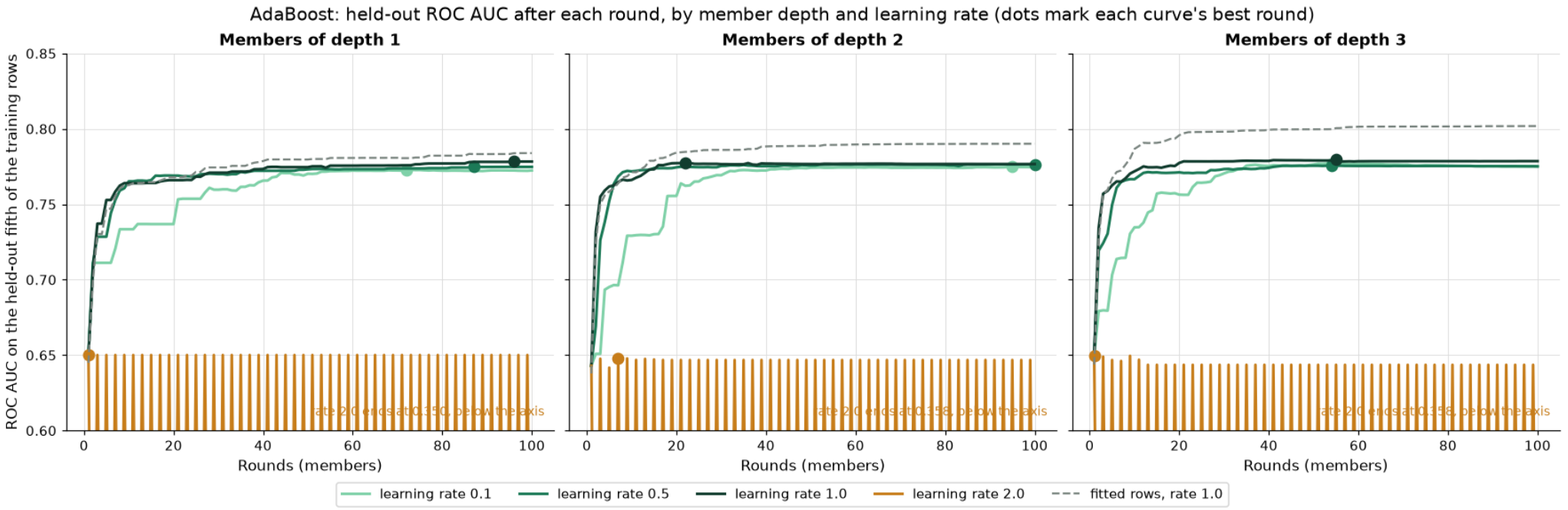
Read it by group

The member decides how much a round can learn. n_estimators and learning_rate decide the number and size of the steps. random_state is housekeeping. No algorithm setting in this version.

Tune first

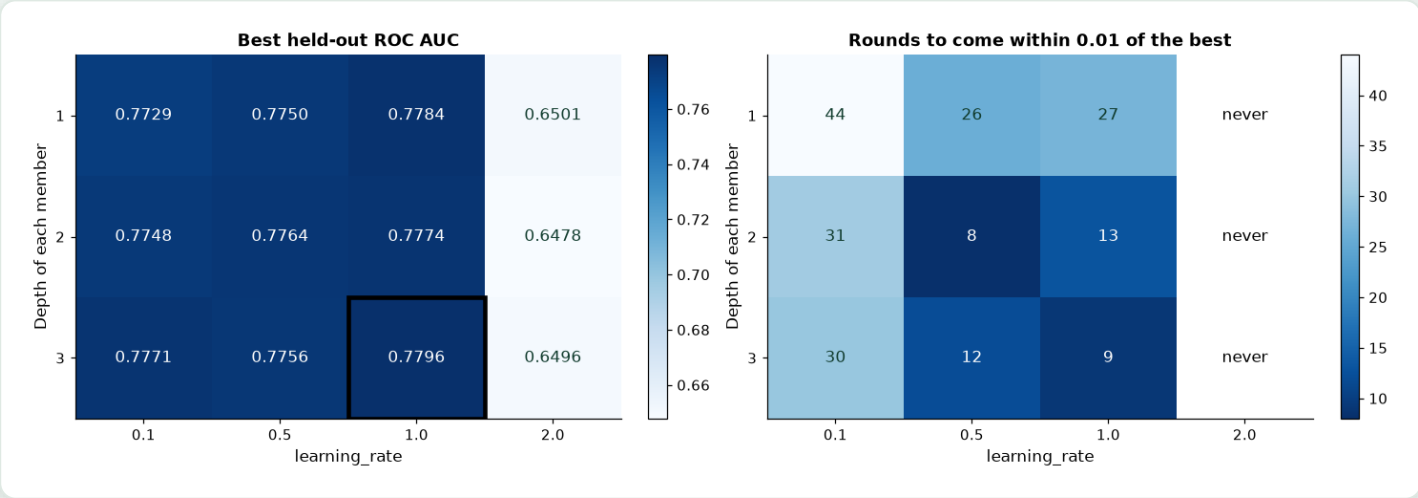
The member's depth, n_estimators, and learning_rate. Days 11 to 14 search the shaded rows. Class weight on the member fights AdaBoost's own reweighting (exercise 4).

Depth and learning rate, side by side

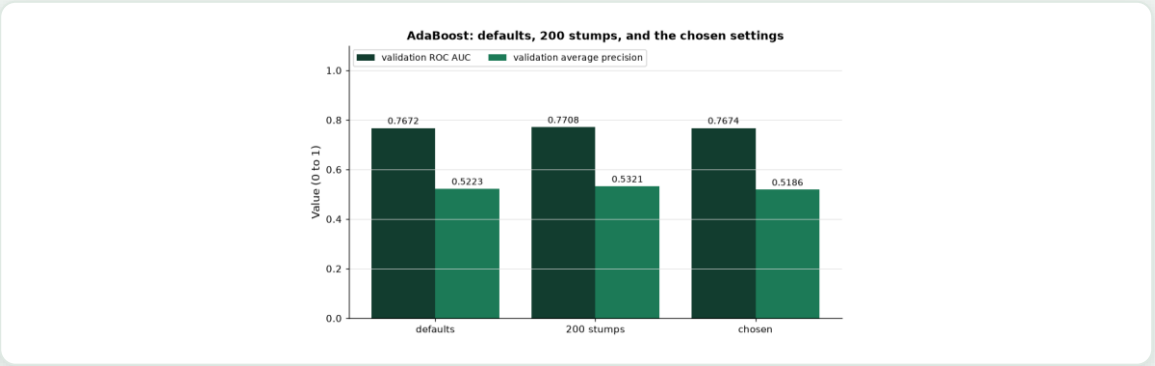


Deeper members memorise more and gain nothing held out. Rates 0.1 to 1.0 end level; a rate of 2.0 overshoots and ranks backwards.

Two settings together, and does the choice carry over



Validation rows	AUC	AP
defaults, 50 stumps	0.7672	0.5223
200 stumps	0.7708	0.5321
chosen: 3, 1.0, 55 rounds	0.7674	0.5186



The held-out fold could not separate settings this close. The defaults and a long run of stumps are already near what AdaBoost can do here.

Where the week's name comes from

Yoav Freund and Robert E. Schapire, August 1997

"A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting"

Journal of Computer and System Sciences, volume 55, issue 1, pages 119 to 139

The last word of the title is the name of this week.

Source, read 5 October 2026: the Crossref record of the paper (DOI 10.1006/jcss.1997.1504). Bibliographic facts only; the paper itself was not read for this course.

Excel side by side

Excel side by side (tested in Excel)

`=1/ROWS($A$2:$A$18001)`	starting weights, D2 down
`=SUMPRODUCT(D2:D18001,A2:A18001)/SUM(D2:D18001)`	weighted error 0.1791
`=LN((1-M2)/M2)`	stump weight 1.5228
`=D2*EXP($M$3*A2)`	missed rows grow, E2 down
`=E2/SUM($E$2:$E$18001)`	new weights, F2 down
`=SUMPRODUCT(F2:F18001,A2:A18001)`	missed rows' share 0.5000
`the same on F and H with B and C`	rounds 2, 3: 0.3526, 0.2592

No AdaBoost in Excel; the stumps' missed columns come from Python. LOG is base 10: =LOG(4) gave 0.6021, =LN(4) gave 1.3863.

Save the day: git stash

1. Commit

mlkit.py, ensemblekit, tests

2. Log

results.csv at the out-of-fold threshold

3. Stash

a learning-rate experiment, put aside

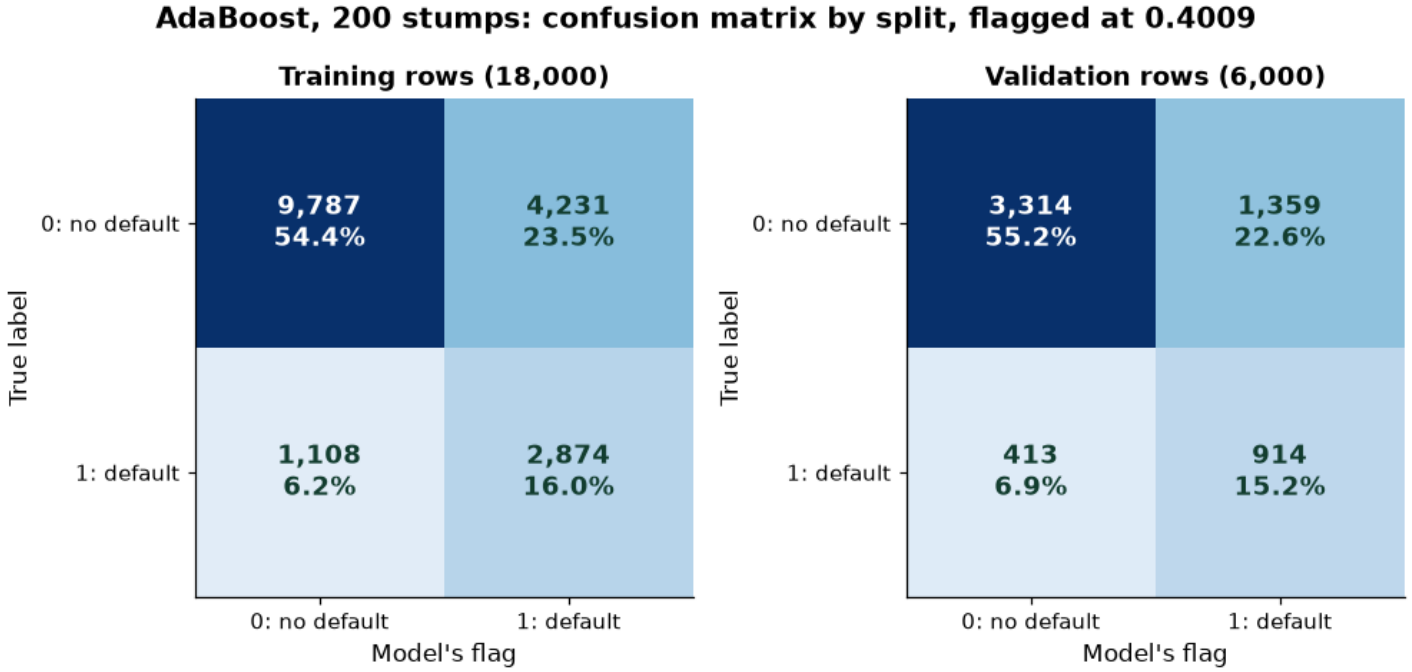
4. Drop

compare, then throw it away

```
model,rows,roc_auc,average_precision,threshold,accuracy,precision,recall,f1
adaboost_200_stumps,validation,0.7708,0.5321,0.4009,0.7047,0.4021,0.6888,0.5078
+adaboost_200_stumps_rate_0.5,validation,0.7706,0.5327,0.3164,0.6980,0.3955,0.6918,0.5033
```

Threshold 0.4009, from out-of-fold scores, far from 0.5. Rate 0.5 gives 0.7706: stashed and dropped.

The standard view at the out-of-fold threshold



Rows	Recall	Precision
training	0.7217	0.4045
validation	0.6888	0.4021

Recall on the training rows and on the validation rows sits close: the panels look alike, as the learning curve said.

Flagged at 0.4009, from out-of-fold scores on the training rows, never at 0.5.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Which rows fit, which rows choose, which rows report: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the stump's weight is computed with np.log10: positive, plausible, and a learning rate nobody chose.

Code that runs is not the same as code that is right.

What you can do now

- Boosting: trees one after another, a weighted vote
- Run a round by hand: error, samme_alpha, reweight
- The missed rows end with half the weight
- Compare settings inside the training rows
- AdaBoost's output is the model's score

Week 2, Day 2 Gradient boosting from residuals, on the benchmark bonds